SENTINEL GRID

Autonomous AI Video Analytics & Threat Fusion Platform for Border CCTV Surveillance

Smart India Hackathon 2026 | Problem Statement: **PS-26187** | Version: **1.0 Production Prototype**

1. Executive & Non-Technical Documentation

1.1 Problem Statement & Background

Securing hundreds of kilometers of national border sectors and tactical perimeters presents critical operational challenges. Human operators monitoring dozens of CCTV and thermal screens suffer a **90% drop in vigilance after 20–30 minutes**. Simultaneously, conventional motion-sensor algorithms generate hundreds of false alarms per day from swaying trees, shadows, wind, and wildlife. Crucially, legacy systems lack **directional intelligence** (distinguishing an intruder entering a zone vs. someone turning back) and produce arbitrary black-box risk scores without natural language explanations.

1.2 The Sentinel Grid Solution

Sentinel Grid transforms standard CCTV, thermal, and PTZ cameras into an autonomous tactical perimeter defense matrix. The system continuously scans live streams, tracks moving persons and vehicles, evaluates virtual fence perimeters, reads vehicle license plates, and calculates an **explainable, trust-weighted threat assessment score (0–100)** with natural language narratives.

1.3 Core Capabilities & Operational Value

Feature / Capability	What It Does	Operational Impact
Multi-Target Tracking	YOLOv8 + ByteTrack persistent tracking for persons, cars, trucks, motorcycles.	Retains target ID identity across temporary visual occlusions.
Directional Fence Breach	Distinguishes between Inbound Intrusions (85-100% Critical Alert) and Outbound Retreats (15-25% Silent Audit).	Eliminates false alarms from friendly patrols and retreating individuals.
Drag-and-Drop Perimeter Calibration	Operators drag and reshape virtual polygon corners directly on live dashboard video tiles.	Zero code needed; tactical officers align perimeters in seconds.
Edge ANPR Plate Reader	Automated crop, bilateral filtering, and EasyOCR alphanumeric character recognition.	Audits authorized vs. unauthorized vehicular ingress.
Explainable Risk Engine	Transparent mathematical formulation combining breach, dwell time, group size, and night multipliers.	Provides plain-English explanations for rapid military/police dispatch.
Night-Vision CLAHE Enhancer	Adaptive local contrast equalization on the CIE LAB Lightness channel.	Enhances low-light and infrared camera footage dynamically.

2. Complete Technical Specifications

2.1 System Architecture & Data Flow

Sentinel Grid operates as a 6-tier pipeline: **1. Ingestion Layer** (Simulated RTSP / Video Source with automatic 640x480 normalization) → **2. Perception Layer** (CLAHE Contrast Enhancer + YOLOv8 Nano Perception) → **3. Tracking Layer** (ByteTrack Multi-Object Tracker with bottom-center ground coordinates) → **4. Spatial & Semantic Analytics** (Shapely Polygon Point-in-Polygon + EasyOCR ANPR) → **5. Trust-Weighted Fusion Engine** (Composite Scorer + Natural Language Narrative Generator) → **6. Persistence & Real-time Distribution** (SQLite Database + FastAPI WebSockets to React Dashboard).

2.2 Mathematical Formulation of Risk Scoring

The composite risk score **R ∈ [0, 100]** for any tracked identity is defined as:

```
R = min(100, max(0, S_base × M_night + S_vehicle))

Where:
• S_base = W_breach + W_dwell + W_group + P_confidence
• W_breach = 85.0 (Inbound Intrusion) | 20.0 (Outbound Safe Retreat) | 0.0 (No Breach)
• W_dwell = min(20.0, (t_zone_dwell / 5.0) × 5.0) [Increments every 5s]
• W_group = (N_nearby - 1) × 8.0 [Proximity radius: 120px]
• P_confidence = -10.0 if YOLO confidence < 0.45, else 0.0
• M_night = 1.25 if night-mode active and target in breach state, else 1.00
• S_vehicle = +5.0 if ANPR license plate registered
```

2.3 Directional Crossing Logic: Inbound vs. Outbound

Using Shapely geometry point containment and vector analysis $\Delta v = (x_t - x_{t-1}, y_t - y_{t-1})$:

- **Case A: Inbound Intrusion (Safe → Restricted)**: Triggers **CRITICAL ALARM (85–100% Risk)**, pulsing red bounding box, and instant WebSocket dispatch of base64 JPEG snapshot.
- **Case B: Outbound Retreat (Restricted → Safe)**: De-escalates threat to **LOW (15–25% Risk)**. Does NOT sound emergency sirens; silently logs crossing timestamps to SQLite tables *breach_events* and *tracks* for audit.

2.4 SQLite Database Schema

Table Name	Primary Columns & Types	Purpose / Retention
tracks	track_id (INT), camera_id (TEXT), class_name (TEXT), confidence (FLOAT), total_dwell_time (FLOAT), max_risk_score (FLOAT), plate_number (TEXT)	Persistent track registry and behavioral statistics.
breach_events	camera_id (TEXT), track_id (INT), fence_name (TEXT), direction ('inbound'/'outbound'), timestamp (DATETIME), position_x, position_y	Audit trail of all physical and virtual boundary transitions.
alerts	alert_id (UUID), camera_id (TEXT), risk_score (FLOAT), severity ('CRITICAL'/'HIGH'), explanation (TEXT), snapshot_base64 (TEXT), acknowledged (BOOL)	Historical security alerts with evidence snapshots.

3. Deployment & Operational Matrix

3.1 Prototype vs. Full Production Comparison

Pillar	Prototype (Hackathon MVP)	Production Tactical Deployment
Ingestion Protocol	Looped .mp4 video files / Synthetic generator	Hardened RTSP / ONVIF Profile S/T/G over fiber / tactical radio mesh
Sensors	RGB Optical + Simulated Night Vision	Long-Range Cooled Mid-Wave Infrared (MWIR) FLIR + Low-Light Starvis
Compute Edge	Multi-threaded CPU / Standard Docker	NVIDIA Jetson AGX Orin Industrial (275 TOPS, MIL-STD-810G ruggedized)
Acceleration	PyTorch CPU Inference (torch.inference_mode)	TensorRT FP16 / INT8 quantized execution (>120 FPS per core)
Persistence	SQLite local file storage (sentinel.db)	Distributed PostgreSQL / TimescaleDB cluster + MinIO S3 object storage
Networking	Local WebSockets & REST API	Secure WebSockets (WSS), MQTT broker, and MIL-STD-188 tactical mesh

3.2 Quick Launch Commands

```
# Option A: Single-Command Docker Deployment
docker compose up --build

# Option B: Local Native Launch
# 1. Backend Service (FastAPI + YOLOv8 + ByteTrack):
cd "backend" && .\venv\Scripts\activate
uvicorn app.main:app --host 0.0.0.0 --port 8000

# 2. Frontend Command Center (React 18 + Vite):
cd "frontend" && npm run dev

# URLs: Dashboard -> http://localhost:3000 | API Docs -> http://localhost:8000/docs
```